

Essential Linux & Command Line Skills for SOC Analysts

Objective: In this activity I used Linux Command-Line skills for log analysis, threat hunting, and incident response automation use by Security Operation Center (SOC) Analysts using a file containing Production-scale sample logs.

Introduction

This lab focused on using Linux command-line tools such as `grep`, `awk`, `sed`, and Regular Expressions (RegEx) for SOC operations. The tasks involved analyzing logs to detect failed logins, suspicious activities, web exploits, and other potential security threats.

Tools

Grep: use for searching patterns in text files

Awk: use for extracting specific fields from log lines

Sed: use for substitution of complex text

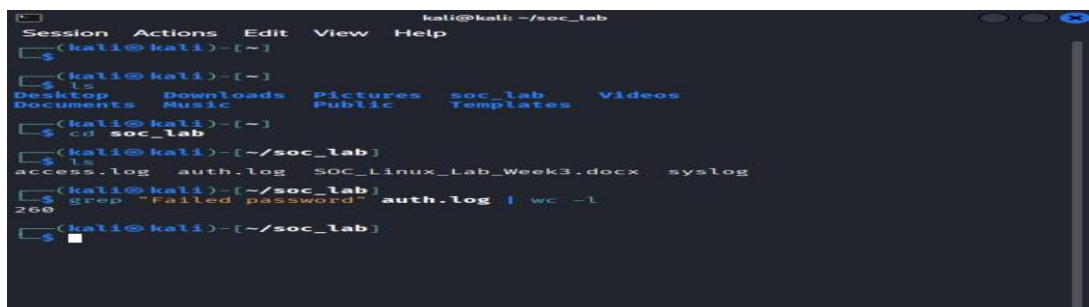
RegEx: use to find complex patterns like C2 beaconing or web exploits.

1: Finding Failed Logins with `grep`

Task: Use `grep` to find all failed SSH login attempts in the `auth.log` file and count them.

I used the following command to carry out the task.

```
grep "Failed password" auth.log | wc -l
```

A screenshot of a terminal window with a dark background. The window title is 'kali@kali: ~/soc_lab'. The prompt is '(kali@kali)-[~]'. The user enters '\$ ls' and the output shows a directory listing: 'ls Desktop Downloads Pictures soc_lab Videos Documents Music Public Templates'. The user enters '\$ cd soc_lab' and the prompt changes to '(kali@kali)-[~/soc_lab]'. The user enters '\$ ls' and the output shows: 'ls access.log auth.log SOC_Linux_Lab_Week3.docx syslog'. The user enters '\$ grep "Failed password" auth.log | wc -l' and the output is '260'. The prompt returns to '(kali@kali)-[~/soc_lab]'.

```
kali@kali: ~/soc_lab
(kali@kali)-[~]
$ ls
Desktop  Downloads  Pictures  soc_lab  Videos
Documents Music      Public    Templates
$ cd soc_lab
(kali@kali)-[~/soc_lab]
$ ls
access.log  auth.log  SOC_Linux_Lab_Week3.docx  syslog
(kali@kali)-[~/soc_lab]
$ grep "Failed password" auth.log | wc -l
260
(kali@kali)-[~/soc_lab]
$
```

Using 'grep' I was able to identify the number of failed login attempt which is **260** attempts

2. Extracting Data with `awk`

Task: Extract only the IP addresses from the failed login attempts you found in Exercise 1, and count how many times each IP address failed to log in.

```
(kali@kali)-[~/soc_lab]
$ grep "Failed password" auth.log | awk '{print $11}' | sort | uniq -c |
sort -nr
    80 198.51.100.22
    60 192.0.2.100
    23 admin
    20 oracle
    20 10.0.0.5
    16 root
    16 mysql
    13 postgres
    12 test

(kali@kali)-[~/soc_lab]
$
```

I used the following command to carry out this task.

```
grep "Failed password" auth.log | awk '{print $11}' | sort | uniq -c | sort -nr
```

Using the `awk` tool I identified that the IP Address with highest failed login attempt is 198.51.100.22

3. Sanitizing logs with `sed`

Task: Use sed to replace the username "admin" with "REDACTEDUSER" in the `auth.log` file, and output the result to a new file called `authsanitized.log`

I used the following command to carry out this task. sed

```
's/admin/REDACTED_USER/g' auth.log > auth_sanitized.log
```

```
(kali@kali)-[~/soc_lab]
$ sed 's/admin/REDACTED_USER/g' auth.log > auth_sanitized.log

(kali@kali)-[~/soc_lab]
$ cat auth_sanitized.log
May 12 06:00:00 webserver sshd[1000]: Accepted publickey for monitoring from
192.168.1.100 port 57202 ssh2
May 12 06:02:00 webserver sshd[1001]: Accepted publickey for monitoring from
192.168.1.51 port 50767 ssh2
May 12 06:04:00 webserver sshd[1002]: Accepted publickey for REDACTED_USER fr
om 192.168.1.50 port 60065 ssh2
May 12 06:06:00 webserver sshd[1003]: Accepted publickey for REDACTED_USER fr
om 192.168.1.100 port 59462 ssh2
May 12 06:08:00 webserver sshd[1004]: Accepted publickey for devops from 192.
168.1.51 port 64496 ssh2
May 12 06:10:00 webserver sshd[1005]: Accepted publickey for REDACTED_USER fr
om 192.168.1.100 port 53001 ssh2
May 12 06:12:00 webserver sshd[1006]: Accepted publickey for sysREDACTED_USER
from 192.168.1.50 port 53328 ssh2
May 12 06:14:00 webserver sshd[1007]: Accepted publickey for sysREDACTED_USER
from 192.168.1.100 port 53822 ssh2
May 12 06:16:00 webserver sshd[1008]: Accepted publickey for sysREDACTED_USER
from 192.168.1.51 port 55838 ssh2
```

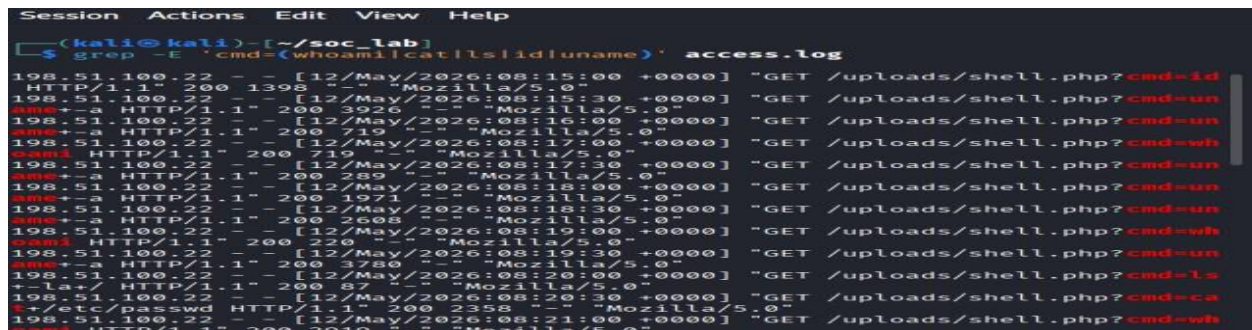
4. Detecting Web Exploits

Scenario: You suspect an attacker has uploaded a web shell to your server.

Task: Use grep with extended regular expressions (-E) to search the access.log for common web shell commands (e.g., whoami, cat /etc/passwd, ls -la).

I used the following command to carry out this task.

```
grep -E 'cmd=(whoami|cat|ls|id|uname)' access.log
```



```
Session Actions Edit View Help
(kali@kali)-[~/soc_lab]
$ grep -E 'cmd=(whoami|cat|ls|id|uname)' access.log
198.51.100.22 - - [12/May/2026:08:15:00 +0000] "GET /uploads/shell.php?cmd=id
HTTP/1.1" 200 1398 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:15:30 +0000] "GET /uploads/shell.php?cmd=un
+ a HTTP/1.1" 200 3926 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:16:00 +0000] "GET /uploads/shell.php?cmd=un
+ a HTTP/1.1" 200 719 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:17:00 +0000] "GET /uploads/shell.php?cmd=wh
+ a HTTP/1.1" 200 719 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:17:30 +0000] "GET /uploads/shell.php?cmd=un
+ a HTTP/1.1" 200 289 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:18:00 +0000] "GET /uploads/shell.php?cmd=un
+ a HTTP/1.1" 200 1971 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:18:30 +0000] "GET /uploads/shell.php?cmd=un
+ a HTTP/1.1" 200 2608 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:19:00 +0000] "GET /uploads/shell.php?cmd=wh
+ a HTTP/1.1" 200 220 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:19:30 +0000] "GET /uploads/shell.php?cmd=un
+ a HTTP/1.1" 200 3780 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:20:00 +0000] "GET /uploads/shell.php?cmd=ls
+ la/ HTTP/1.1" 200 87 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:20:30 +0000] "GET /uploads/shell.php?cmd=ca
+ /etc/passwd HTTP/1.1" 200 2358 "-" "Mozilla/5.0"
198.51.100.22 - - [12/May/2026:08:21:00 +0000] "GET /uploads/shell.php?cmd=wh
+ a HTTP/1.1" 200 2919 "-" "Mozilla/5.0"
```

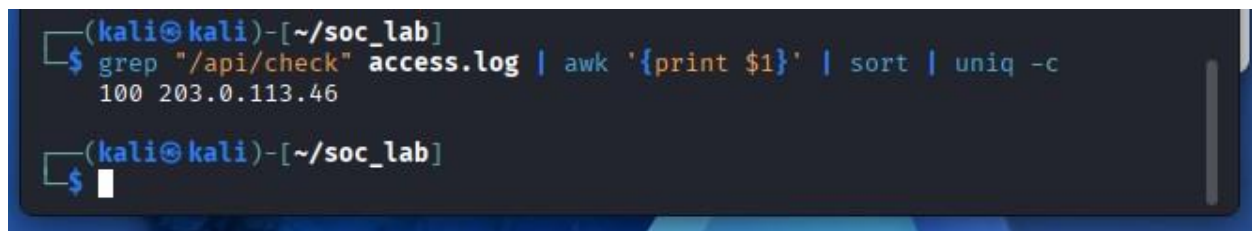
I searched the access.log file for any log entries containing a cmd= parameter that attempts to run common system commands like whoami, cat, ls, id, or uname, which may indicate command injection or web shell activity.

5. Detecting C2 Beaconing

Task: Search the access.log for repeated requests to the same API endpoint that might indicate beaconing.

I used the following command to carry out this task. grep

```
"/api/check" access.log | awk '{print $1}' | sort | uniq -c
```



```
(kali@kali)-[~/soc_lab]
$ grep "/api/check" access.log | awk '{print $1}' | sort | uniq -c
100 203.0.113.46
(kali@kali)-[~/soc_lab]
$
```

This command searches the access.log file for requests to the /api/check endpoint, extracts the source IP addresses from those requests, sorts them, and counts how many times each IP accessed that endpoint.

6. Correlating Events (The Soc Mindset)

It's important to bear in mind that failed logins alone may not be malicious, but a failed login followed by a successful is regarded as a high risk.

Task: Look at the auth.log. Can you identify an IP address that had multiple failed logins followed by a successful login?

I used the following command to carry out this task.

```
grep "Accepted password" auth.log
```

```
(kali@kali)-[~/soc_lab]
$ grep "Accepted password" auth.log
May 12 07:20:00 webserver sshd[3000]: Accepted password for admin from 203.0.113.45 port 38921 ssh2

(kali@kali)-[~/soc_lab]
$
```

I identify that the attacker with the IP Address 203.0.113.45 successfully brute-forced an account through SSH.

INCIDENT RESPONSE CHALLENGE

Based on your previous findings, you know that multiple attackers targeted your infrastructure. You need to determine the full scope of the compromise

Task: The Web Vector

Did the attacker at 198.51.100.22 access the web server before or after their SSH brute force attempts?

I used the following command to carry out this task.

```
grep "198.51.100.22" access.log | head -n 5
```

```
(kali@kali)-[~/soc_lab]
$ grep "198.51.100.22" access.log | head -n 5
198.51.100.22 - - [12/May/2026:07:40:00 +0000] "POST /login HTTP/1.1" 200 881
"- "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
198.51.100.22 - - [12/May/2026:07:40:15 +0000] "POST /login HTTP/1.1" 200 894
"- "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
198.51.100.22 - - [12/May/2026:07:40:30 +0000] "POST /login HTTP/1.1" 401 551
"- "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
198.51.100.22 - - [12/May/2026:07:40:45 +0000] "POST /login HTTP/1.1" 401 751
"- "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
198.51.100.22 - - [12/May/2026:07:41:00 +0000] "POST /login HTTP/1.1" 401 878
"- "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"

(kali@kali)-[~/soc_lab]
$
```

This command searches the `access.log` file for entries containing the IP address 198.51.100.22 and displays only the first 5 matching log entries.

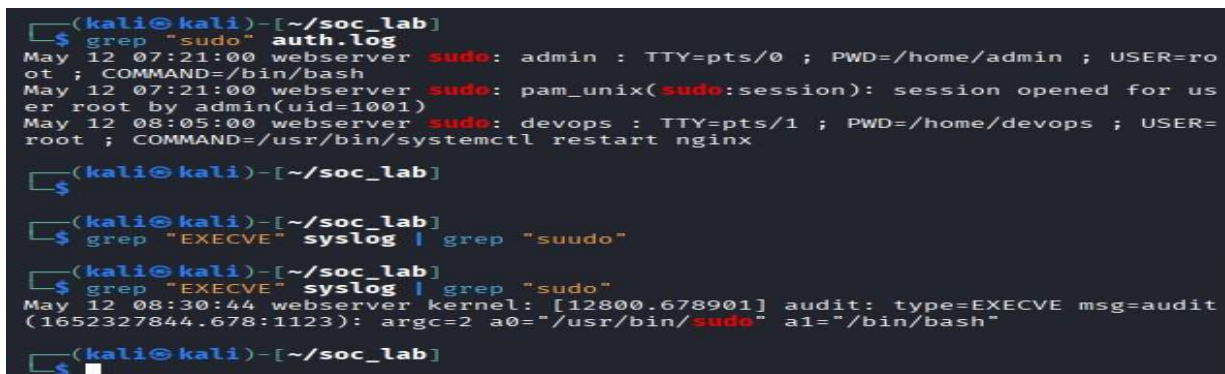
In this task I identified that attacker at 198.51.100.22 accessed the web server after their successful SSH brute force attack.

Task 2: Privilege Escalation

After compromising the SSH admin account, did the attacker escalate privileges? Check both `auth.log` and `syslog`.

I used the following command to carry out this task.

```
grep "sudo" auth.log  
grep "EXECVE" syslog | grep "sudo"
```



The terminal screenshot shows the following commands and output:

```
(kali@kali)-[~/soc_lab]  
$ grep "sudo" auth.log  
May 12 07:21:00 webserver sudo: admin : TTY=pts/0 ; PWD=/home/admin ; USER=root ; COMMAND=/bin/bash  
May 12 07:21:00 webserver sudo: pam_unix(sudo:session): session opened for user root by admin(uid=1001)  
May 12 08:05:00 webserver sudo: devops : TTY=pts/1 ; PWD=/home/devops ; USER=root ; COMMAND=/usr/bin/systemctl restart nginx  
  
(kali@kali)-[~/soc_lab]  
$  
  
(kali@kali)-[~/soc_lab]  
$ grep "EXECVE" syslog | grep "sudo"  
  
(kali@kali)-[~/soc_lab]  
$ grep "EXECVE" syslog | grep "sudo"  
May 12 08:30:44 webserver kernel: [12800.678901] audit: type=EXECVE msg=audit(1652327844.678:1123): argc=2 a0="/usr/bin/sudo" a1="/bin/bash"  
  
(kali@kali)-[~/soc_lab]  
$
```

These logs confirm successful privilege escalation. The compromised admin account used `sudo` to launch `/bin/bash` as root, and the authentication logs show a root session was successfully opened.

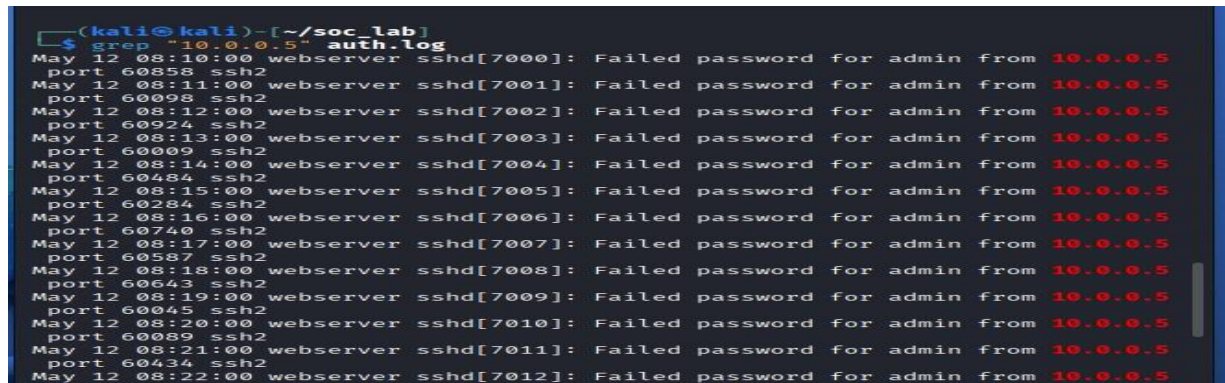
The Audit logs in `syslog` show execution of `/usr/bin/sudo /bin/bash`, indicating the attacker used `sudo` to spawn a root shell after compromising the admin account. This confirms successful privilege escalation.

Task 3: Lateral Movement

Did any attacker attempt to move laterally to other internal systems (e.g., 10.0.0.5)?

I used the following command to carry out this task.

```
grep "10.0.0.5" auth.log
```



```
(kali@kali)~[~/soc_lab]
$ grep "10.0.0.5" auth.log
May 12 08:10:00 webserver sshd[7000]: Failed password for admin from 10.0.0.5
port 60858 ssh2
May 12 08:11:00 webserver sshd[7001]: Failed password for admin from 10.0.0.5
port 60098 ssh2
May 12 08:12:00 webserver sshd[7002]: Failed password for admin from 10.0.0.5
port 60924 ssh2
May 12 08:13:00 webserver sshd[7003]: Failed password for admin from 10.0.0.5
port 60009 ssh2
May 12 08:14:00 webserver sshd[7004]: Failed password for admin from 10.0.0.5
port 60484 ssh2
May 12 08:15:00 webserver sshd[7005]: Failed password for admin from 10.0.0.5
port 60284 ssh2
May 12 08:16:00 webserver sshd[7006]: Failed password for admin from 10.0.0.5
port 60740 ssh2
May 12 08:17:00 webserver sshd[7007]: Failed password for admin from 10.0.0.5
port 60587 ssh2
May 12 08:18:00 webserver sshd[7008]: Failed password for admin from 10.0.0.5
port 60643 ssh2
May 12 08:19:00 webserver sshd[7009]: Failed password for admin from 10.0.0.5
port 60045 ssh2
May 12 08:20:00 webserver sshd[7010]: Failed password for admin from 10.0.0.5
port 60089 ssh2
May 12 08:21:00 webserver sshd[7011]: Failed password for admin from 10.0.0.5
port 60434 ssh2
May 12 08:22:00 webserver sshd[7012]: Failed password for admin from 10.0.0.5
```

The Logs show repeated failed SSH login attempts to the admin account originating from the internal host 10.0.0.5. This suggests attempted lateral movement within the network but no successful authentication events were identified

Conclusion

This lab demonstrated how Linux command-line tools such as grep, awk, sed, and RegEx can help SOC analysts detect and investigate security threats. Through log analysis, I was able to identify failed logins, suspicious activities, privilege escalation, and attempted lateral movement, showing the importance of Linux skills in cybersecurity operations.

Report Prepared By:

Joseph Emeka Ani